Revision 20–23 · Serving metrics, load, percentiles & SLOs

Interactive revision of lesson 20, lesson 21, lesson 22 and lesson 23: one question at a time, logged in my own reasoning.

Q1 · Two tools, one run, two TPOTs PARTIAL

Covers: 20

> **QUESTION**
>
> A load test ran 50 short chats (20 output tokens each, 50 ms between tokens) and 10 long answers (1,000 output tokens each, 10 ms between tokens). The dashboard reports an average TPOT of about 13.5 ms; a teammate's script reports about 43 ms. Compute both averages, say which method each tool used, and pick the number to quote to a PM as "what a typical user feels".

How I solved it

1. Each request's TPOT is the average gap inside it: every short chat has 50 ms, every long answer 10 ms.
2. **Per request** (each request gets one vote, and the vote is its TPOT):

(50 × 50 ms + 10 × 10 ms) ÷ 60 = 2,600 ÷ 60 ≈ **43.3 ms**. That's the teammate's script.

1. **Per token** (each gap gets one vote). A reply with N tokens has N − 1 gaps:

	Requests	Gaps each	Total gaps	Gap time
Short chats	50	19	950	950 × 50 ms = 47,500 ms
Long answers	10	999	9,990	9,990 × 10 ms = 99,900 ms
Total	60		**10,940**	**147,400 ms**

147,400 ÷ 10,940 ≈ **13.5 ms**. That's the dashboard.

1. The long answers own 9,990 of the 10,940 gaps (91%), so they swamp the per-token average.
2. For "what a typical user feels", quote **43.3 ms (per request)**: each user counts once, and 50 of the 60 users (83%) saw 50 ms gaps.

Neither number is wrong, but a report must say which average it uses. Better still, segment: "short chats 50 ms, long answers 10 ms".

> **RE-DRILL**
>
> Re-drill: build the gap table (requests × gaps each) before dividing, and sanity-check that the per-request average sits near the majority's value.

Q2 · The stream looks slower but feels faster SOLID

Covers: 20

QUESTION

After turning on speculative decoding, a client script that times the gaps between streamed network messages reports TPOT = 45 ms (it was 20 ms before). Users say the stream feels faster. Each message now carries 3 tokens on average. What is the script measuring wrong, and what is the real TPOT?

How I solved it

1. The server now sends several tokens per network message, so the script measures the gap between **messages**, not between tokens.
2. One message every 45 ms carrying 3 tokens: 45 ÷ 3 = **15 ms per token**.
3. 15 ms beats the old 20 ms: about 25% faster (≈ 67 vs 50 tokens/s), which matches what users feel.

The fix is to count tokens, not messages: TPOT = (E2E − TTFT) ÷ (N − 1), with N from the token count. Message gaps remain useful for spotting stalls.

RE-DRILL

Re-drill: whenever a measurement changes after a server-side feature flips, ask what unit the timer is counting.

Q3 · "Request X is 4× slower" SKIPPED

Covers: 20, 08

QUESTION

Request X: client E2E 8.3 s, 400 output tokens. Request Y: 2.0 s, 85 output tokens. Both have server TTFT 300 ms and TPOT 20 ms. The PM says X is 4× slower. Is it, and what should the PM compare instead?

How it's solved

1. E2E ≈ TTFT + TPOT × (N − 1).
2. X: 0.3 + 0.02 × 399 = 0.3 + 7.98 ≈ **8.3 s**. Y: 0.3 + 0.02 × 84 = 0.3 + 1.68 ≈ **2.0 s**.
3. Both match the formula with identical TTFT and TPOT: X isn't slower, it wrote ~4.7× more tokens.
4. Compare **TTFT and TPOT**, or E2E only between replies of similar length.

RE-DRILL

Re-drill: before calling a request slow on E2E, divide out its length.

Q4 · The benchmark said we're fine PARTIAL

Covers: 21

QUESTION

A closed-loop benchmark with 32 concurrent users measured a mean time in system of 4 s. At launch, real users will arrive at 10 req/s. How many req/s did the benchmark serve, what happens at launch, and why didn't the test warn us?

How I solved it

1. Little's law: $L = \lambda W$, so $\lambda = L \div W = 32 \div 4\ s =$ **8 req/s** served.
2. At launch, arrivals (10 req/s) exceed completions ($\approx$ 8 req/s), so the queue grows by $10 - 8 =$ **2 req/s**.
3. After one minute: $2 \times 60 =$ **120 requests waiting**, on top of the 32 in service, and it never stops growing: every new user waits longer than the last.
4. The closed loop hid it: its 32 users wait for each reply before sending again, so a slow server just gets fewer requests. Real users are an open loop and keep arriving.

RE-DRILL

Re-drill: convert a concurrency test to req/s with Little's law, then compare it with the expected arrival rate.

Q5 · The marketing campaign PARTIAL

Covers: 21

QUESTION

A server runs at 70% busy with a fine P99. A campaign adds 30% more traffic. Using time in system = service ÷ (1 − utilization), what is the new utilization and how much longer does a request spend in the system?

How I solved it

1. Utilization = arrival rate × service time. Service time is unchanged, so utilization scales with traffic: $0.70 \times 1.3 =$ **0.91**.
2. Time in system as a multiple of service time:

	Utilization	$1 \div (1 - \rho)$
Before	0.70	$1 \div 0.30 \approx 3.33$
After	0.91	$1 \div 0.09 \approx 11.1$

1. $11.1 \div 3.33 \approx$ **3.3× longer**, from only 30% more traffic.

Real traffic is bursty, so the tail gets hit even harder than this idealized queue: leave headroom instead of running at 90%+.

> **RE-DRILL**
>
> Re-drill: new ρ = old ρ × traffic factor, then compare $1 \div (1 - \rho)$ before and after.

Q6 · Pick the right load test PARTIAL

Covers: 21

> **QUESTION**
>
> Two tasks: (a) find the maximum tokens/s one GPU delivers; (b) check P99 TTFT stays under 1 s at the launch traffic of 10 req/s. Which load generator and arrival pattern for each?

Answer

- **(a) Max throughput:** closed loop, **sweep concurrency** 1 → 2 → 4 → … until tokens/s stops growing. No request-rate sweep needed: the closed loop already keeps the GPU as full as the concurrency allows.
- **(b) Latency target:** **open loop at 10 req/s with Poisson arrivals**, closest to many independent users. Then repeat with bursts (burstiness < 1), since production has them.
- Always report the **achieved** request rate next to the configured one: if they differ, the server couldn't keep up.

> **RE-DRILL**
>
> Re-drill: match the question to the knob (capacity → concurrency, user experience → arrival rate).

Q7 · The stall your load test didn't see SKIPPED

Covers: 22

> **QUESTION**
>
> A load generator meant to send 5 req/s waits for each response. Normal latency is 40 ms; once a minute the server stalls for 3 s. How many requests should land in the stall, and does P99 catch it for the waiting client vs a scheduled one?

How to think it through

Step 0 · What P99 can see. P99 is the latency that 99% of requests beat. It only "sees" the stall if **more than 1%** of the recorded requests are slow. So the whole question is: what share of recorded requests is slow, for each client?

Step 1 · Count what should have been sent. Work in one stall cycle, one minute:

5 req/s × 60 s = **300 intended requests**, one every 1 ÷ 5 = **0.2 s**

Why the 40 ms latency doesn't change this: a schedule fires every 0.2 s regardless of how long replies take. Even the waiting client keeps pace in normal times, because each 40 ms reply returns well before the next 0.2 s slot. It only falls behind when a reply takes longer than 0.2 s, which happens during the stall. (The 40 ms also adds to stalled waits, 3.04 s instead of 3.0 s: negligible.)

Step 2 · Count what lands in the stall. The stall lasts 3 s, and the schedule keeps firing every 0.2 s during it:

5 req/s × 3 s = **15 requests**, all slow

Step 3 · Scheduled client: compare with 1%.

15 ÷ 300 = **5%** slow. 5% > 1%, so P99 lands **inside the stall**.

Step 4 · How slow is that P99? Time each request from when it was *meant* to be sent. A request scheduled at the start of the stall waits the full 3 s; one scheduled near the end waits almost nothing:

```
time into stall:  0.0  0.2  0.4  0.6  ...  2.6  2.8   (s)
waits:            3.0  2.8  2.6  2.4  ...  0.4  0.2   (s)
```

The slowest 1% of 300 is 3 requests: 3.0, 2.8 and 2.6 s. So scheduled **P99 ≈ 2.6 s**.

Step 5 · Waiting client: what does it actually record? It sends one request at the start of the stall, then sits blocked for 3 s, so it never sends the other 14:

```
scheduled:  | | | | | |[x x x x x x x x x x x x x x x]| | | | |   15 slow
waiting:    | | | | | |[x . . . . . . . . . . . . . .]| | | | |    1 slow
                        └──────── 3 s stall ──────────┘
```

Recorded per minute ≈ 300 − 14 = 286, of which 1 is slow: 1 ÷ 286 ≈ **0.35%**. Under 1%, so **P99 ≈ 40 ms**: the stall is invisible.

Step 6 · Name it and fix it. The missing 14 are exactly the ones that would have been slow: **coordinated omission**. Fix: send on a schedule, time from the **intended send time**, and count timeouts as slow.

	Recorded	Slow	Share	P99
Waiting client	~286	1	0.35%	≈ 40 ms
Scheduled, from intended send	300	15	5%	≈ 2.6 s

RE-DRILL

Re-drill: compare "slow requests ÷ intended requests" with 1% to see whether P99 can see a stall.

Q8 · P99 doubled, nothing changed PARTIAL

Covers: 22, 15

> **QUESTION**
>
> P99 TTFT went from 0.8 s last week to 1.5 s this week. No deploys, and the engine's per-request timings look unchanged. What's the likely explanation, and how do you prove it?

Answer

- **The traffic mix shifted:** more long-prompt requests (documents, RAG) this week. TTFT is driven by **prompt length** (prefill), so a bigger long-prompt share drags the blended P99 up even though no request got slower.
- **Prove it by segmenting:** split by prompt length (also model, cache hit vs miss), take P99 **inside each group**, and compare week over week. Each group's P99 is unchanged while the long-prompt **share** rose.

> **RE-DRILL**
>
> Re-drill: before blaming the server for a percentile jump, check whether the population changed.

Q9 · The fleet P99 PARTIAL

Covers: 22

> **QUESTION**
>
> Four replicas with equal traffic report P99 TTFT of 0.6, 0.7, 0.6 and 3.0 s. Ops averages them into "fleet P99 = 1.2 s". Why is that wrong, is the true value higher or lower, and what data do you need?

How I solved it

1. Percentiles don't average. Keep each replica's **full distribution as a histogram**, merge the histograms, and take the percentile last.
2. Higher or lower? Take 100,000 fleet requests: the fleet P99 is set by the slowest **1,000**.
3. Replica 4 serves **25,000** and has by far the worst tail, so nearly all 1,000 slow requests come from it: 1,000 ÷ 25,000 = **4%** of its requests.
4. So fleet P99 ≈ **replica 4's P96**. With a P99 of 3.0 s and a tail that heavy, its P96 is likely around 2 s or more: **well above 1.2 s**.

The average flatters: three good replicas dilute one bad one, but the true percentile is decided by the bad one.

> **RE-DRILL**
>
> Re-drill: fleet tail share ÷ bad replica's traffic share = which of its percentiles sets the fleet P99.

Q10 · What's the goodput? PARTIAL

Covers: 23

QUESTION

SLO: TTFT < 500 ms AND TPOT < 30 ms. At 6 req/s offered, 5% fail with 429; among successes, 90% meet TTFT and 85% meet TPOT (independent). The dashboard shows 6 req/s throughput. What's the goodput, and what share of the dashboard number is good?

How I solved it

A good request passes three gates: it didn't fail, it met TTFT, and it met TPOT.

1. Remove failures (a 429 is a bad request): $6 \times 0.95 =$ **5.7 req/s** succeed.
2. "AND" of independent targets multiplies: $0.90 \times 0.85 =$ **0.765** of successes meet both.
3. Goodput $= 5.7 \times 0.765 \approx$ **4.4 req/s**.
4. Share of the dashboard: $4.4 \div 6 \approx$ **73%** good; the other 27% is rejections (5%) plus slow requests.

RE-DRILL

Re-drill: list the gates a request must pass, multiply their pass rates, and never drop failures.

Q11 · Should someone be paged? PARTIAL

Covers: 23

QUESTION

SLO: 99.5% good over a 30-day month, at about 2 million requests a month. Over the last hour, 3% of requests were bad. How big is the error budget, what's the burn rate, and how many days until it's gone?

How I solved it

1. Budget = share allowed to be bad × requests: $0.5\% \times 2{,}000{,}000 = 2 \times 10^6 \times 5 \times 10^{-3} =$ **10,000 bad requests**.
2. Burn rate = bad rate now ÷ allowed bad rate = 3% ÷ 0.5% = **6×**.
3. Days left = 30 days ÷ burn rate = 30 ÷ 6 = **5 days**.

RE-DRILL

Re-drill: anchor on "1× burn = the whole window" before dividing.